

中任意位置的 k 个任意的物理页面。由于页表工作的方式，操作系统没有必要分配 k 个连续的物理内存页面。页面可以随机地分散在物理内存中。

9.5 虚拟内存作为内存保护的工具体

任何现代计算机系统必须为操作系统提供手段来控制对内存系统的访问。不应该允许一个用户进程修改它的只读代码段。而且也不应该允许它读或修改任何内核中的代码和数据结构。不应该允许它读或者写其他进程的私有内存，并且不允许它修改任何与其他进程共享的虚拟页面，除非所有的共享者都显式地允许它这么做(通过调用明确的进程间通信系统调用)。

就像我们所看到的，提供独立的地址空间使得区分不同进程的私有内存变得容易。但是，地址翻译机制可以以一种自然的方式扩展到提供更好的访问控制。因为每次 CPU 生成一个地址时，地址翻译硬件都会读一个 PTE，所以通过在 PTE 上添加一些额外的许可位来控制对一个虚拟页面内容的访问十分简单。图 9-10 展示了大致的思想。

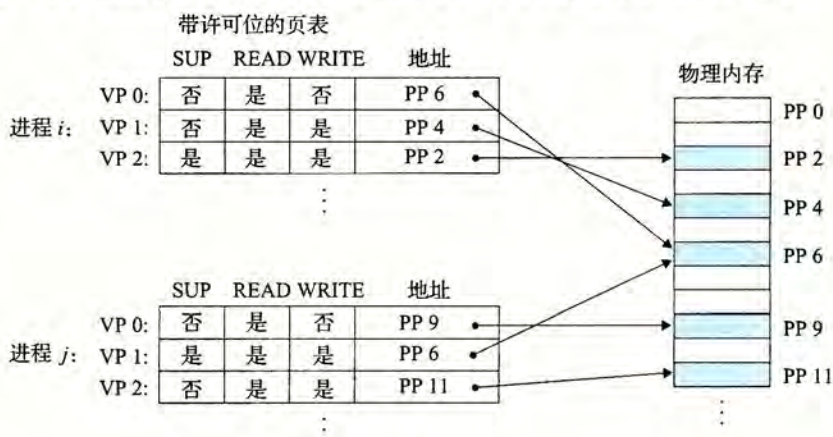


图 9-10 用虚拟内存来提供页面级的内存保护

在这个示例中，每个 PTE 中已经添加了三个许可位。SUP 位表示进程是否必须运行在内核(超级用户)模式下才能访问该页。运行在内核模式中的进程可以访问任何页面，但是运行在用户模式中的进程只允许访问那些 SUP 为 0 的页面。READ 位和 WRITE 位控制对页面的读和写访问。例如，如果进程 i 运行在用户模式下，那么它有读 VP 0 和读写 VP 1 的权限。然而，不允许它访问 VP 2。

如果一条指令违反了这些许可条件，那么 CPU 就触发一个一般保护故障，将控制传递给一个内核中的异常处理程序。Linux shell 一般将这种异常报告为“段错误(segmentation fault)”。

9.6 地址翻译

这一节讲述的是地址翻译的基础知识。我们的目标是让你了解硬件在支持虚拟内存中的角色，并给出足够多的细节使得你可以亲手演示一些具体的示例。不过，要记住我们省略了大量的细节，尤其是和时序相关的细节，虽然这些细节对硬件设计者来说是非常重要的，但是超出了我们讨论的范围。图 9-11 概括了我们在这一节里将要使用的所有符号，供读者参考。

基本参数	
符 号	描 述
$N = 2^n$	虚拟地址空间中的地址数量
$M = 2^m$	物理地址空间中的地址数量
$P = 2^p$	页的大小（字节）

虚拟地址（VA）的组成部分	
符 号	描 述
VPO	虚拟页面偏移量（字节）
VPN	虚拟页号
TLBI	TLB 索引
TLBT	TLB 标记

物理地址（PA）的组成部分	
符 号	描 述
PPO	物理页面偏移量（字节）
PPN	物理页号
CO	缓冲块内的字节偏移量
CI	高速缓存索引
CT	高速缓存标记

图 9-11 地址翻译符号小结

形式上来说，地址翻译是一个 N 元素的虚拟地址空间(VAS)中的元素和一个 M 元素的物理地址空间(PAS)中元素之间的映射，

$$\text{MAP}: \text{VAS} \rightarrow \text{PAS} \cup \varnothing$$

这里

$$\text{MAP}(A) = \begin{cases} A' & \text{如果虚拟地址 } A \text{ 处的数据在 PAS 的物理地址 } A' \text{ 处} \\ \varnothing & \text{如果虚拟地址 } A \text{ 处的数据不在物理内存中} \end{cases}$$

图 9-12 展示了 MMU 如何利用页表来实现这种映射。CPU 中的一个控制寄存器，页表基址寄存器(Page Table Base Register, PTBR)指向当前页表。 n 位的虚拟地址包含两个部分：一个 p 位的虚拟页面偏移(Virtual Page Offset, VPO)和一个 $(n-p)$ 位的虚拟页号(Virtual

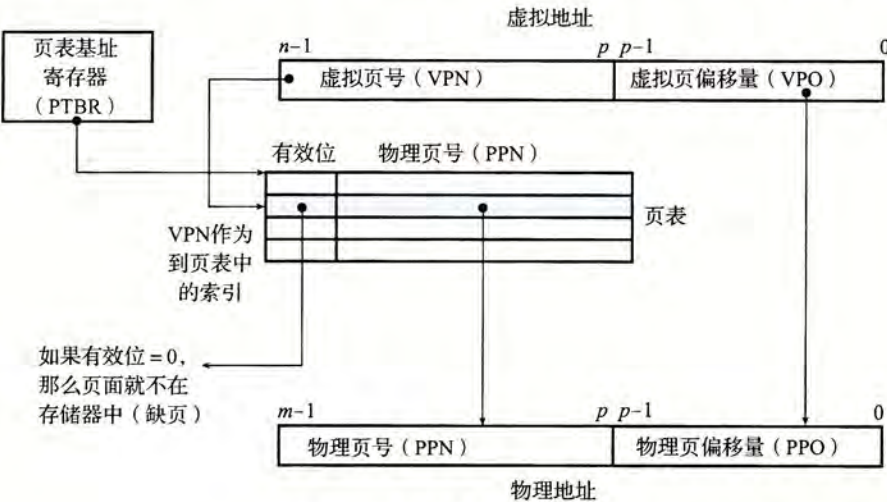


图 9-12 使用页表的地址翻译

Page Number, VPN)。MMU 利用 VPN 来选择适当的 PTE。例如, VPN 0 选择 PTE 0, VPN 1 选择 PTE 1, 以此类推。将页表条目中物理页号(Physical Page Number, PPN)和虚拟地址中的 VPO 串联起来, 就得到相应的物理地址。注意, 因为物理和虚拟页面都是 P 字节的, 所以物理页面偏移(Physical Page Offset, PPO)和 VPO 是相同的。

图 9-13a 展示了当页面命中时, CPU 硬件执行的步骤。

- 第 1 步: 处理器生成一个虚拟地址, 并把它传送给 MMU。
- 第 2 步: MMU 生成 PTE 地址, 并从高速缓存/主存请求得到它。
- 第 3 步: 高速缓存/主存向 MMU 返回 PTE。
- 第 4 步: MMU 构造物理地址, 并把它传送给高速缓存/主存。
- 第 5 步: 高速缓存/主存返回所请求的数据字给处理器。

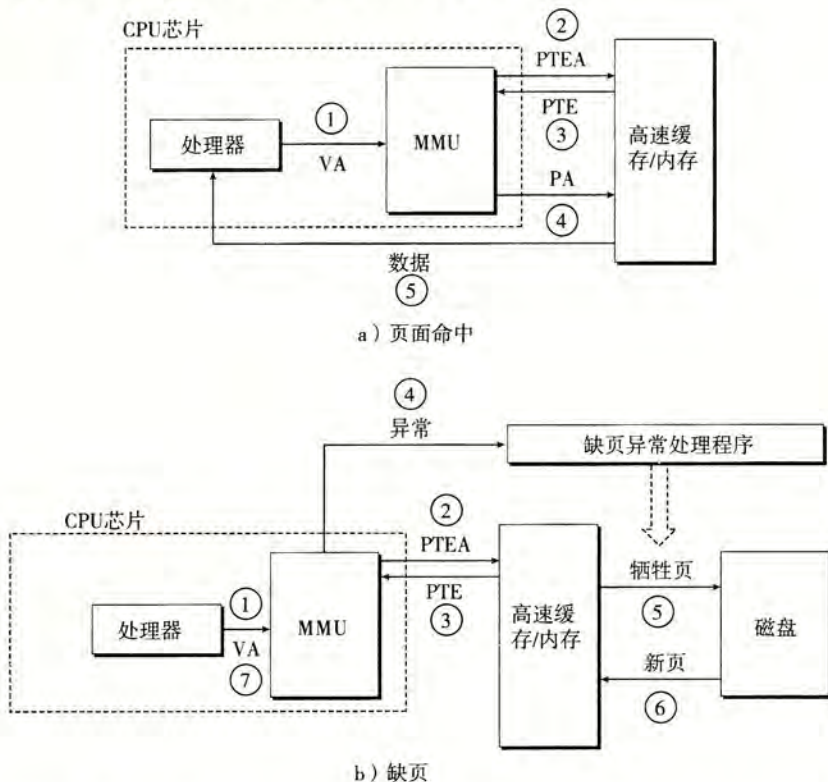


图 9-13 页面命中和缺页的操作图(VA: 虚拟地址。PTEA: 页表条目地址。PTE: 页表条目。PA: 物理地址)

页面命中完全是由硬件来处理的, 与之不同的是, 处理缺页要求硬件和操作系统内核协作完成, 如图 9-13b 所示。

- 第 1 步到第 3 步: 和图 9-13a 中的第 1 步到第 3 步相同。
- 第 4 步: PTE 中的有效位是零, 所以 MMU 触发了一次异常, 传递 CPU 中的控制到操作系统内核中的缺页异常处理程序。
- 第 5 步: 缺页处理程序确定出物理内存中的牺牲页, 如果这个页面已经被修改了, 则把它换出到磁盘。
- 第 6 步: 缺页处理程序页面调入新的页面, 并更新内存中的 PTE。

- 第 7 步：缺页处理程序返回到原来的进程，再次执行导致缺页的指令。CPU 将引起缺页的虚拟地址重新发送给 MMU。因为虚拟页面现在缓存在物理内存中，所以就会命中，在 MMU 执行了图 9-13b 中的步骤之后，主存就会将所请求字返回给处理器。

 **练习题 9.3** 给定一个 32 位的虚拟地址空间和一个 24 位的物理地址，对于下面的页面大小 P ，确定 VPN、VPO、PPN 和 PPO 中的位数：

P	VPN位数	VPO位数	PPN位数	PPO位数
1KB				
2KB				
4KB				
8KB				

9.6.1 结合高速缓存和虚拟内存

在任何既使用虚拟内存又使用 SRAM 高速缓存的系统中，都有应该使用虚拟地址还是使用物理地址来访问 SRAM 高速缓存的问题。尽管关于这个折中的详细讨论已经超出了我们的讨论范围，但是大多数系统是选择物理寻址的。使用物理寻址，多个进程同时在高速缓存中有存储块和共享来自相同虚拟页面的块成为很简单的事情。而且，高速缓存无需处理保护问题，因为访问权限的检查是地址翻译过程的一部分。

图 9-14 展示了一个物理寻址的高速缓存如何和虚拟内存结合起来。主要的思路是地址翻译发生在高速缓存查找之前。注意，页表条目可以缓存，就像其他的数据字一样。

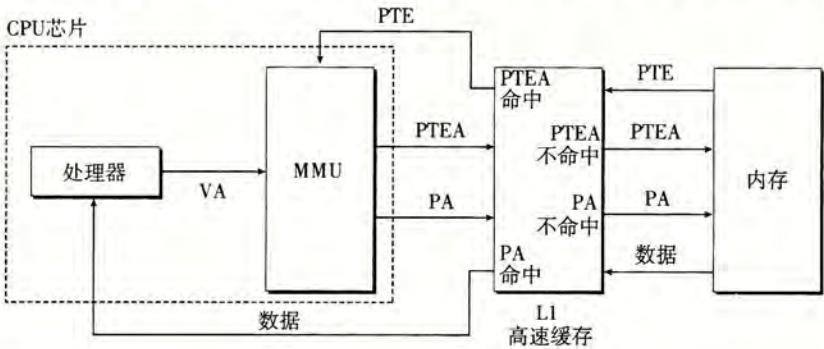


图 9-14 将 VM 与物理寻址的高速缓存结合起来(VA：虚拟地址。
PTEA：页表条目地址。PTE：页表条目。PA：物理地址)

9.6.2 利用 TLB 加速地址翻译

正如我们看到的，每次 CPU 产生一个虚拟地址，MMU 就必须查阅一个 PTE，以便将虚拟地址翻译为物理地址。在最糟糕的情况下，这会要求从内存多取一次数据，代价是几十到几百个周期。如果 PTE 碰巧缓存在 L1 中，那么开销就下降到 1 个或 2 个周期。然而，许多系统都试图消除即使是这样的开销，它们在 MMU 中包括了一个关于 PTE 的小的缓存，称为翻译后备缓冲器(Translation Lookaside Buffer, TLB)。

TLB 是一个小的、虚拟寻址的缓存，其中每一行都保存着一个由单个 PTE 组成的块。TLB 通常有高度的相联度。如图 9-15 所示，用于组选择和行匹配的索引和标记字段是从

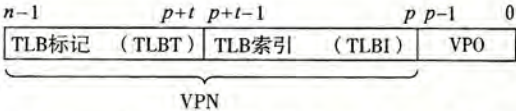


图 9-15 虚拟地址中用以访问 TLB 的组成部分

虚拟地址中的虚拟页号中提取出来的。如果 TLB 有 $T=2^t$ 个组, 那么 TLB 索引 (TLBI) 是由 VPN 的 t 个最低位组成的, 而 TLB 标记 (TLBT) 是由 VPN 中剩余的位组成的。

图 9-16a 展示了当 TLB 命中时 (通常情况) 所包括的步骤。这里的关键点是, 所有的地址翻译步骤都是在芯片上的 MMU 中执行的, 因此非常快。

- 第 1 步: CPU 产生一个虚拟地址。
- 第 2 步和第 3 步: MMU 从 TLB 中取出相应的 PTE。
- 第 4 步: MMU 将这个虚拟地址翻译成一个物理地址, 并且将它发送到高速缓存/主存。
- 第 5 步: 高速缓存/主存将所请求的数据字返回给 CPU。

当 TLB 不命中时, MMU 必须从 L1 缓存中取出相应的 PTE, 如图 9-16b 所示。新取出的 PTE 存放在 TLB 中, 可能会覆盖一个已经存在的条目。

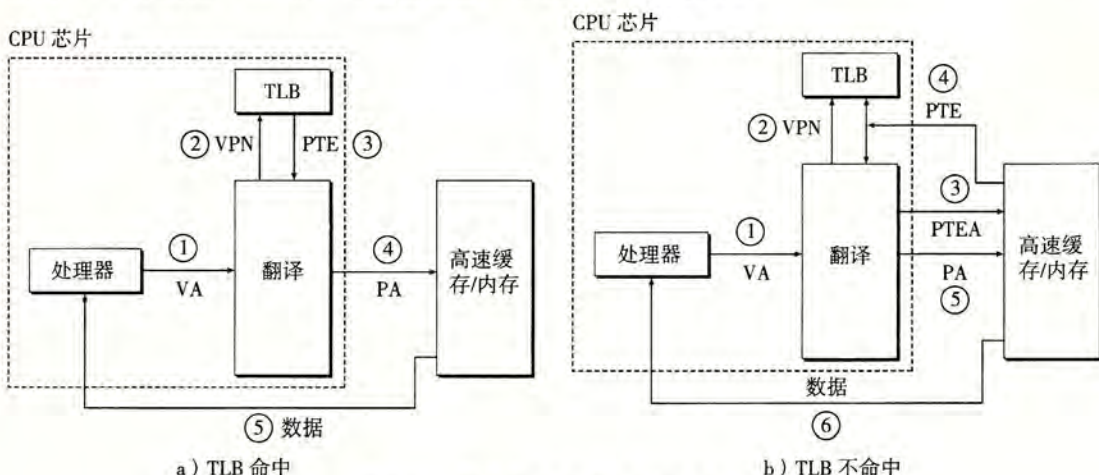


图 9-16 TLB 命中和不命中的操作图

9.6.3 多级页表

到目前为止, 我们一直假设系统只用一个单独的页表来进行地址翻译。但是如果有一个 32 位的地址空间、4KB 的页面和一个 4 字节的 PTE, 那么即使应用所引用的只是虚拟地址空间中很小的一部分, 也总是需要一个 4MB 的页表驻留在内存中。对于地址空间为 64 位的系统来说, 问题将变得更复杂。

用来压缩页表的常用方法是使用层次结构的页表。用一个具体的示例是最容易理解这个思想的。假设 32 位虚拟地址空间被分为 4KB 的页, 而每个页表条目都是 4 字节。还假设在这一时刻, 虚拟地址空间有如下形式: 内存的前 2K 个页面分配给了代码和数据, 接下来的 6K 个页面还未分配, 再接下来的 1023 个页面也未分配, 接下来的 1 个页面分配给了用户栈。图 9-17 展示了我们如何为这个虚拟地址空间构造一个两级的页表层次结构。

一级页表中的每个 PTE 负责映射虚拟地址空间中一个 4MB 的片(chunk), 这里每一片都是由 1024 个连续的页面组成的。比如, PTE 0 映射第一片, PTE 1 映射接下来的一片, 以此类推。假设地址空间是 4GB, 1024 个 PTE 已经足够覆盖整个空间了。

如果片 i 中的每个页面都未被分配, 那么一级 PTE i 就为空。例如, 图 9-17 中, 片 2~7 是未被分配的。然而, 如果在片 i 中至少有一个页是分配了的, 那么一级 PTE i 就指向一个二级页表的基址。例如, 在图 9-17 中, 片 0、1 和 8 的所有或者部分已被分配, 所以它们的一级 PTE 就指向二级页表。

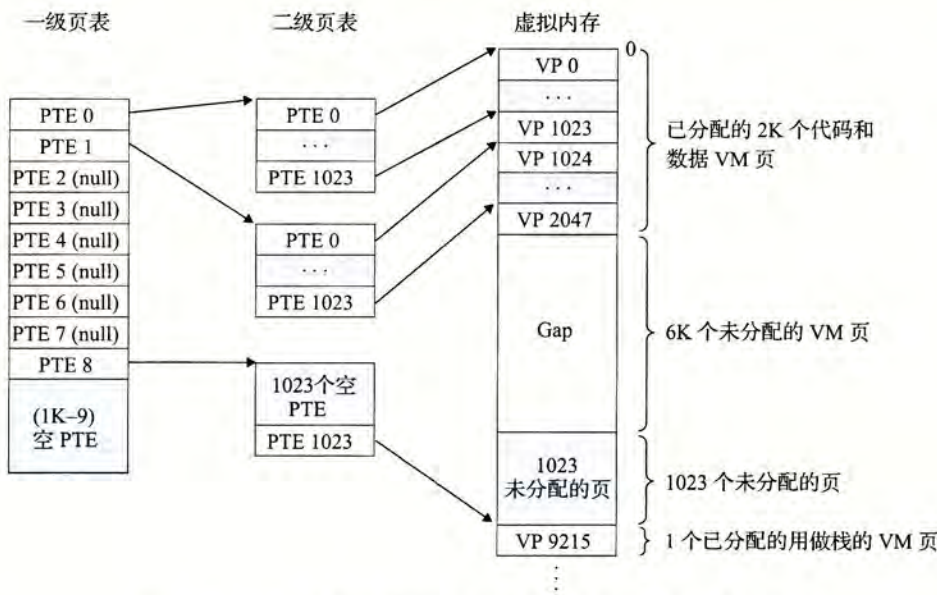


图 9-17 一个两级页表层次结构。注意地址是从上往下增加的

二级页表中的每个 PTE 都负责映射一个 4KB 的虚拟内存页面，就像我们查看只有一级的页表一样。注意，使用 4 字节的 PTE，每个一级和二级页表都是 4KB 字节，这刚好和一个页面的大小是一样的。

这种方法从两个方面减少了内存要求。第一，如果一级页表中的一个 PTE 是空的，那么相应的二级页表就根本不会存在。这代表着一种巨大的潜在节约，因为对于一个典型的程序，4GB 的虚拟地址空间的大部分都会是未分配的。第二，只有一级页表才需要总是在主存中；虚拟内存系统可以在需要时创建、页面调入或调出二级页表，这就减少了主存的压力；只有最经常使用的二级页表才需要缓存在主存中。

图 9-18 描述了使用 k 级页表层次结构的地址翻译。虚拟地址被划分成为 k 个 VPN 和 1 个 VPO。每个 VPN i 都是一个到第 i 级页表的索引，其中 $1 \leq i \leq k$ 。第 j 级页表中的每个 PTE， $1 \leq j \leq k-1$ ，都指向第 $j+1$ 级的某个页表的基址。第 k 级页表中的每个 PTE 包含某个物理页面的 PPN，或者一个磁盘块的地址。为了构造物理地址，在能够确定 PPN 之前，MMU 必须访问 k 个 PTE。对于只有一级的页表结构，PPO 和 VPO 是相同的。

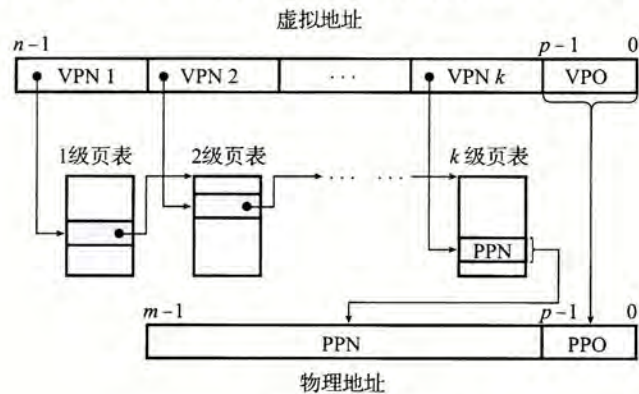


图 9-18 使用 k 级页表的地址翻译

访问 k 个 PTE，第一眼看上去昂贵而不切实际。然而，这里 TLB 能够起作用，正是通过将不同层次上页表的 PTE 缓存起来。实际上，带多级页表的地址翻译并不比单级页表慢很多。

9.6.4 综合：端到端的地址翻译

在这一节里，我们通过一个具体的端到端的地址翻译示例，来综合一下我们刚学过的这些内容，这个示例运行在有一个 TLB 和 L1 d-cache 的小系统上。为了保证可管理性，我们做出如下假设：

- 内存是按字节寻址的。
- 内存访问是针对 1 字节的字的(不是 4 字节的字)。
- 虚拟地址是 14 位长的($n=14$)。
- 物理地址是 12 位长的($m=12$)。
- 页面大小是 64 字节($P=64$)。
- TLB 是四路组相联的，总共有 16 个条目。
- L1 d-cache 是物理寻址、直接映射的，行大小为 4 字节，而总共有 16 个组。

图 9-19 展示了虚拟地址和物理地址的格式。因为每个页面是 $2^6=64$ 字节，所以虚拟地址和物理地址的低 6 位分别作为 VPO 和 PPO。虚拟地址的高 8 位作为 VPN。物理地址的高 6 位作为 PPN。

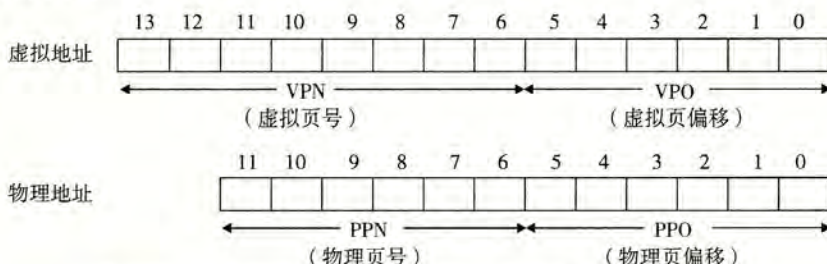
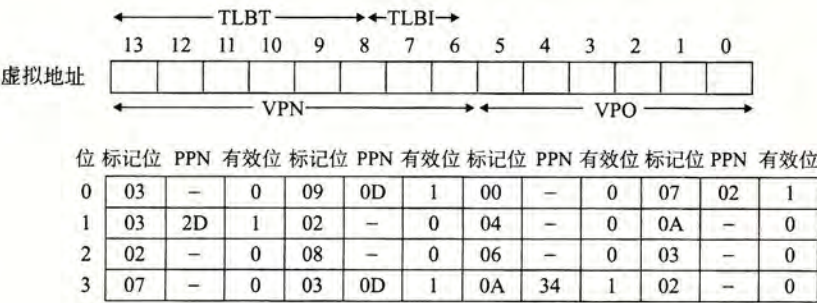


图 9-19 小内存系统的寻址。假设 14 位的虚拟地址($n=14$)，12 位的物理地址($m=12$)和 64 字节的页面($P=64$)

图 9-20 展示了小内存系统的一个快照，包括 TLB(图 9-20a)、页表的一部分(图 9-20b)和 L1 高速缓存(图 9-20c)。在 TLB 和高速缓存的图上面，我们还展示了访问这些设备时硬件是如何划分虚拟地址和物理地址的位的。

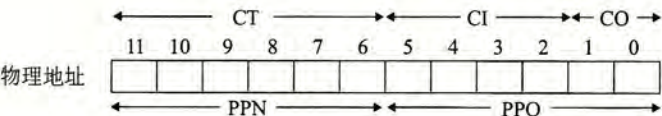
- TLB。TLB 是利用 VPN 的位进行虚拟寻址的。因为 TLB 有 4 个组，所以 VPN 的低 2 位就作为组索引(TLBI)。VPN 中剩下的高 6 位作为标记(TLBT)，用来区别可能映射到同一个 TLB 组的不同的 VPN。
- 页表。这个页表是一个单级设计，一共有 $2^8=256$ 个页表条目(PTE)。然而，我们只对这些条目中的开头 16 个感兴趣。为了方便，我们用索引它的 VPN 来标识每个 PTE；但是要记住这些 VPN 并不是页表的一部分，也不储存在内存中。另外，注意每个无效 PTE 的 PPN 都用一个破折号来表示，以加强一个概念：无论刚好这里存储的是什么位值，都是没有任何意义的。
- 高速缓存。直接映射的缓存是通过物理地址中的字段来寻址的。因为每个块都是 4 字节，所以物理地址的低 2 位作为块偏移(CO)。因为有 16 组，所以接下来的 4 位就用来表示组索引(CI)。剩下的 6 位作为标记(CT)。



a) TLB：四组，16 个条目，四路组相联

VPN	PPN	有效位	VPN	PPN	有效位
00	28	1	08	13	1
01	—	0	09	17	1
02	33	1	0A	09	1
03	02	1	0B	—	0
04	—	0	0C	—	0
05	16	1	0D	2D	1
06	—	0	0E	11	1
07	—	0	0F	0D	1

b) 页表：只展示了前 16 个 PTE



索引	标记位	有效位	块 0	块 1	块 2	块 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	08
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

c) 高速缓存：16 个组，4 字节的块，直接映射

图 9-20 小内存系统的 TLB、页表以及缓存。TLB、页表和缓存中所有的值都是十六进制表示的

给定了这种初始化设定，让我们来看看当 CPU 执行一条读地址 0x03d4 处字节的加载指令时会发生什么。（回想一下我们假定 CPU 读取 1 字节的字，而不是 4 字节的字。）为了

开始这种手工的模拟，我们发现写下虚拟地址的各个位，标识出我们会需要的各种字段，并确定它们的十六进制值，是非常有帮助的。当硬件解码地址时，它也执行相似的任务。

	TLBT						TLBI							
	0x03						0x03							
位位置	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VA = 0x03d4	0	0	0	0	1	1	1	1	0	1	0	1	0	0
	VPN						VPO							
	0x0f						0x14							

开始时，MMU 从虚拟地址中抽取出 VPN(0x0F)，并且检查 TLB，看它是否因为前面的某个内存引用缓存了 PTE 0x0F 的一个副本。TLB 从 VPN 中抽取出 TLB 索引(0x03)和 TLB 标记(0x3)，组 0x3 的第二个条目中有效匹配，所以命中，然后将缓存的 PPN(0x0D)返回给 MMU。

如果 TLB 不命中，那么 MMU 就需要从主存中取出相应的 PTE。然而，在这种情况下，我们很幸运，TLB 会命中。现在，MMU 有了形成物理地址所需要的所有东西。它通过将来自 PTE 的 PPN(0x0D)和来自虚拟地址的 VPO(0x14)连接起来，这就形成了物理地址(0x354)。

接下来，MMU 发送物理地址给缓存，缓存从物理地址中抽取出缓存偏移 CO(0x0)、缓存组索引 CI(0x5)以及缓存标记 CT(0x0D)。

	CT						CI				CO	
	0x0d						0x05				0x0	
位位置	11	10	9	8	7	6	5	4	3	2	1	0
PA = 0x354	0	0	1	1	0	1	0	1	0	1	0	0
PPN						PPO						
0x0d						0x14						

因为组 0x5 中的标记与 CT 相匹配，所以缓存检测到一个命中，读出在偏移量 CO 处的数据字节(0x36)，并将它返回给 MMU，随后 MMU 将它传递回 CPU。

翻译过程的其他路径也是可能的。例如，如果 TLB 不命中，那么 MMU 必须从页表中的 PTE 中取出 PPN。如果得到的 PTE 是无效的，那么就产生一个缺页，内核必须调入合适的页面，重新运行这条加载指令。另一种可能性是 PTE 是有效的，但是所需要的内存块在缓存中不命中。

 **练习题 9.4** 说明 9.6.4 节中的示例内存系统是如何将一个虚拟地址翻译成物理地址和访问缓存的。对于给定的虚拟地址，指明访问的 TLB 条目、物理地址和返回的缓存字节值。指出是否发生了 TLB 不命中，是否发生了缺页，以及是否发生了缓存不命中。如果是缓存不命中，在“返回的缓存字节”栏中输入“—”。如果有缺页，则在“PPN”一栏中输入“—”，并且将 C 部分和 D 部分空着。

虚拟地址：0x03d7

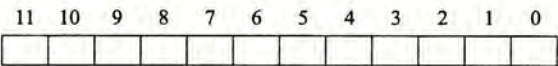
A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0

B. 地址翻译

参数	值
VPN	
TLB索引	
TLB标记	
TLB命中? (是/否)	
缺页? (是/否)	
PPN	

C. 物理地址格式



D. 物理内存引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中? (是/否)	
返回的缓存字节	

9.7 案例研究：Intel Core i7/Linux 内存系统

我们以一个实际系统的案例研究来总结我们对虚拟内存的讨论：一个运行 Linux 的 Intel Core i7。虽然底层的 Haswell 微体系结构允许完全的 64 位虚拟和物理地址空间，而现在的(以及可预见的未来的)Core i7 实现支持 48 位(256TB)虚拟地址空间和 52 位(4PB)物理地址空间，还有一个兼容模式，支持 32 位(4GB)虚拟和物理地址空间。

图 9-21 给出了 Core i7 内存系统的重要部分。处理器封装(processor package)包括四个核、一个大的所有核共享的 L3 高速缓存，以及一个 DDR3 内存控制器。每个核包含一个层次结构的 TLB、一个层次结构的数据和指令高速缓存，以及一组快速的点到点链路，这种链路基于 QuickPath 技术，是为了让一个核与其他核和外部 I/O 桥直接通信。TLB 是虚拟寻址的，是四路组相联的。L1、L2 和 L3 高速缓存是物理寻址的，块大小为 64 字节。L1 和 L2 是 8 路组相联的，而 L3 是 16 路组相联的。页大小可以在启动时被配置为 4KB 或 4MB。Linux 使用的是 4KB 的页。

9.7.1 Core i7 地址翻译

图 9-22 总结了完整的 Core i7 地址翻译过程，从 CPU 产生虚拟地址的时刻一直到来自内存的数据字到达 CPU。Core i7 采用四级页表层次结构。每个进程有它自己私有的页表层次结构。当一个 Linux 进程在运行时，虽然 Core i7 体系结构允许页表换进换出，但是与已分配了的页相关联的页表都是驻留在内存中的。CR3 控制寄存器指向第一级页表(L1)的起始位置。CR3 的值是每个进程上下文的一部分，每次上下文切换时，CR3 的值都会被恢复。